

¿Qué es un “Pipeline”?

Un **Pipeline** en programación es una secuencia de pasos o procesos conectados entre sí, donde la salida de un paso se convierte en la entrada del siguiente. Su objetivo es automatizar tareas, organizar flujos de trabajo y reducir errores manuales.

Ejemplo sencillo

Imagina una fábrica de botellas:

1. Se fabrica la botella.
2. Se llena con agua.
3. Se coloca la tapa.
4. Se etiqueta.
5. Se empaqueta.

Cada etapa recibe el resultado de la anterior. En programación ocurre exactamente lo mismo.

¿Para qué sirve un Pipeline?

Los pipelines se utilizan para:

- Automatizar procesos repetitivos.
- Procesar datos.
- Integrar y desplegar aplicaciones.
- Ejecutar pruebas automáticas.
- Mejorar la eficiencia y la calidad del software.

Pasos para Crear un Pipeline

1. Definir el objetivo

Primero debes identificar qué proceso deseas automatizar.

Ejemplo:

Automatizar la publicación de una aplicación web.

2. Identificar las Etapas

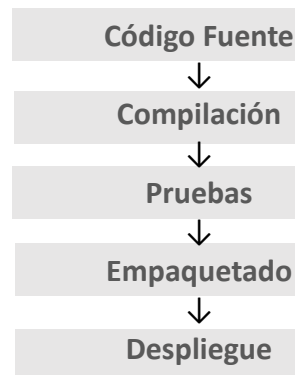
Divide el proceso en pasos independientes.

Ejemplo:

- Descargar código fuente.
- Compilar la aplicación.
- Ejecutar pruebas.
- Generar archivos de despliegue.
- Publicar en el servidor.

3. Establecer el flujo de ejecución

Define el orden en que se ejecutarán los pasos.



4. Automatizar cada Etapa

Utiliza scripts o herramientas que ejecuten automáticamente cada tarea.

Herramientas comunes:

- Jenkins
- GitHub Actions
- GitLab CI/CD
- Azure DevOps
- CircleCI

5. Monitorear resultados

Verifica que cada etapa se complete correctamente y registra errores para su análisis.

Ejemplo Práctico 1: Pipeline de Desarrollo de Software

Cuando un programador sube cambios al repositorio:

Paso 1

Se descarga el código.

Paso 2

Se compila el proyecto.

Paso 3

Se ejecutan pruebas automáticas.

Paso 4

Si las pruebas son exitosas, se genera una versión lista para producción.

Paso 5

La aplicación se despliega automáticamente en el servidor.

Ejemplo Práctico 2: Pipeline de Procesamiento de Datos

Supongamos que una empresa recibe datos de ventas diariamente.

Paso 1

Importar archivos CSV.

Paso 2

Limpiar datos incorrectos.

Paso 3

Transformar formatos de fechas y monedas.

Paso 4

Calcular métricas de ventas.

Paso 5

Guardar resultados en una base de datos.

Ejemplo Práctico 3: Pipeline en Python

```
def obtener_datos():  
    return [1, 2, 3, 4, 5]  
  
def duplicar(datos):  
    return [x * 2 for x in datos]  
  
def filtrar(datos):  
    return [x for x in datos if x > 5]  
  
datos = obtener_datos()  
datos = duplicar(datos)  
datos = filtrar(datos)  
  
print(datos)  
Resultado:  
[6, 8, 10]
```

➤ Cada función representa una etapa del pipeline.

Ventajas de Utilizar Pipelines

- Automatización de tareas.
- Menos errores humanos.
- Mayor velocidad de ejecución.
- Fácil mantenimiento.
- Mejor organización de procesos.
- Escalabilidad.

Conclusión.

Un Pipeline es una cadena de procesos donde cada etapa transforma información o ejecuta una tarea específica antes de pasar el resultado a la siguiente. Es una técnica ampliamente utilizada en desarrollo de software, integración continua (CI/CD), análisis de datos e inteligencia artificial para automatizar y optimizar flujos de trabajo.